LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – ABSTRACT DAN INTERFACE



NAMA: ANDI MUHAMMAD ABID JAYA

NIM: 103132400014

KELAS: S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Teori

Dalam modul ini mempelajari beberapa dasar teori dasar sebagai berikut:

1. Kelas Abstract

   Kelas abstract adalah kelas yang dideklarasikan menggunakan keyword abstract dan tidak dapat dibuat menjadi objek secara langsung. Kelas ini dapat memiliki abstract method yang belum memiliki implementasi dan harus diimplementasikan oleh kelas turunannya. Contohnya, kelas Employee merupakan kelas abstract yang memiliki method earnings() untuk menghitung pendapatan pegawai. Method tersebut diimplementasikan oleh kelas turunannya, seperti SalariedEmployee.

2. Interface

   Interface adalah struktur yang mendefinisikan method atau konstanta yang akan diimplementasikan pada objek lain. Kelas yang menggunakan interface harus mengimplementasikan method abstrak yang dideklarasikan di dalamnya dengan menggunakan keyword implements. Contohnya, interface Payable memiliki method getPaymentAmount() yang diimplementasikan oleh kelas Invoice untuk menghitung total pembayaran berdasarkan jumlah barang dan harga per item.

B. Guided

1. Guided 1

```
// nama file: Employee.java

public abstract class Employee {
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;

    // Constructor
    public Employee(String first, String last, String ssn) {
        firstName = first;
        lastName = last;
        socialSecurityNumber = ssn;
    }

    // Setter dan getter firstName
    public void setFirstName(String first) {
        firstName = first;
    }
```

```
    public String getFirstName() {
        return firstName;
    }

    // Setter dan getter lastName
    public void setLastName(String last) {
        lastName = last;
    }

    public String getLastName() {
        return lastName;
    }

    // Setter dan getter socialSecurityNumber
    public void setSocialSecurityNumber(String ssn) {
        socialSecurityNumber = ssn;
    }

    public String getSocialSecurityNumber() {
        return socialSecurityNumber;
    }

    // Menampilkan informasi Employee
    public String toString() {
        return String.format(
            "%s %s\nsocial security number: %s",
            getFirstName(),
            getLastName(),
            getSocialSecurityNumber()
        );
    }

    // Abstract method
    public abstract double earnings();
}
```

```
//  nama file: SalariedEmployee.java

public class SalariedEmployee extends Employee {

    private double weeklySalary;
```

```
    // Constructor
    public SalariedEmployee(
        String first,
        String last,
        String ssn,
        double salary
    ) {
        super(first, last, ssn);
        setWeeklySalary(salary);
    }

    // Setter weeklySalary
    public void setWeeklySalary(double salary) {
        weeklySalary = salary < 0.0 ? 0.0 : salary;
    }

    // Getter weeklySalary
    public double getWeeklySalary() {
        return weeklySalary;
    }

    // Implementasi abstract method earnings
    @Override
    public double earnings() {
        return getWeeklySalary();
    }

    // Menampilkan informasi SalariedEmployee
    @Override
    public String toString() {
        return String.format(
            "salaried employee: %s\n%s: $%,.2f",
            super.toString(),
            "weekly salary",
            getWeeklySalary()
        );
    }
}
```

```
//  nama file: MainEmployee.java
```

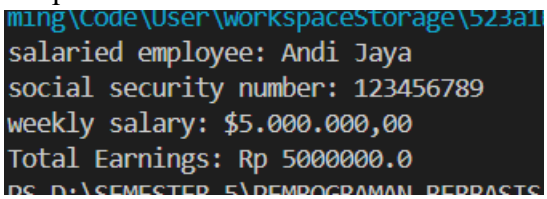
```
public class MainEmployee {

    public static void main(String[] args) {

        SalariedEmployee employee = new SalariedEmployee(
            "Andi",
            "Jaya",
            "123456789",
            5000000
        );

        System.out.println(employee);
        System.out.println("Total Earnings: Rp " + employee.earnings());
    }
}
```

Output :



2. Guided 2

```
//   nama file: Payable.java

package guided.guided2;

public interface Payable {
    double getPaymentAmount();
}
```

```
//  nama file: Invoice.java

package guided.guided2;

public class Invoice implements Payable {
```

```
private String partNumber;
private String partDescription;
private int quantity;
private double pricePerItem;

public Invoice(String part, String description, int count, double price) {
   partNumber = part;
   partDescription = description;
   setQuantity(count);
   setPricePerItem(price);
}

public void setPartNumber(String part) {
   partNumber = part;
}

public String getPartNumber() {
   return partNumber;
}

public void setPartDescription(String description) {
   partDescription = description;
}

public String getPartDescription() {
   return partDescription;
}

public void setQuantity(int count) {
   quantity = (count < 0) ? 0 : count;
}

public int getQuantity() {
   return quantity;
}

public void setPricePerItem(double price) {
   pricePerItem = (price < 0.0) ? 0.0 : price;
}

public double getPricePerItem() {
   return pricePerItem;
```

```
    }

    public String toString() {
        return String.format("%s: \n%s: %s (%s) \n%s: %d \n%s: $%,.2f",
            "invoice", "part number", getPartNumber(),
            getPartDescription(), "quantity", getQuantity(),
            "price per item", getPricePerItem());
    }

    public double getPaymentAmount() {
        return getQuantity() * getPricePerItem();
    }
}
```

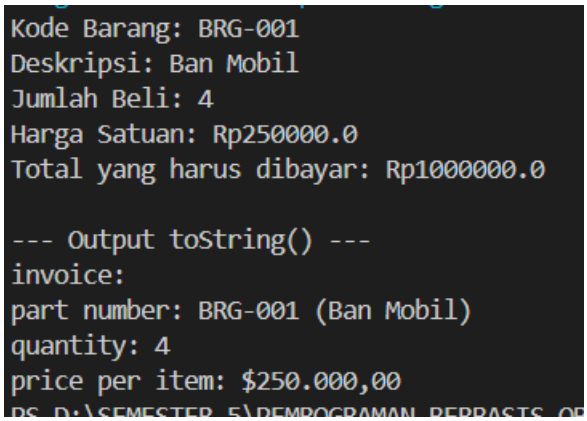
```
//  nama file: MainInvoice.java

package guided.guided2;

public class MainInvoice {
    public static void main(String[] args) {
        Invoice invoice1 = new Invoice("BRG-001", "Ban Mobil", 4, 250000.0);

        System.out.println("Kode Barang: " + invoice1.getPartNumber());
        System.out.println("Deskripsi: " + invoice1.getPartDescription());
        System.out.println("Jumlah Beli: " + invoice1.getQuantity());
        System.out.println("Harga Satuan: Rp" + invoice1.getPricePerItem());
        System.out.println("Total yang harus dibayar: Rp" +
invoice1.getPaymentAmount());
        System.out.println("\n--- Output toString() ---");
        System.out.println(invoice1.toString());
    }
}
```

Output :



3. Guided 3

```
// nama file: MyInterface.java

package guided.guided3;

interface MyInterface {
    default void newMethod() {
        System.out.println("Newly added default method");
    }

    void existingMethod(String str);
}
```

```
// nama file: Example.java

package guided.guided3;

public class Example implements MyInterface {

    public void existingMethod(String str) {
        System.out.println("String is: " + str);
    }

    public static void main(String[] args) {
        Example obj = new Example();
        obj.newMethod();
        obj.existingMethod("Java 8 is easy to learn");
```

```
    }
}
```

Output :

```
Newly added default method
String is: Java 8 is easy to learn
```

C. Unguided

1. Soal Unguided

Soal Unguided — Modul 7: Pemrograman Berorientasi Objek

Abstraksi, Kontrak Interface, dan Polimorfisme Heterogen

Petunjuk Umum

Buatlah program Java berdasarkan skenario sistem kasir bioskop berikut. Rancang kelas dan antarmuka dengan mematuhi prinsip OOP yang telah dipelajari.

- **Struktur Berkas Pengerjaan (`unguided/`)** Seluruh kelas dan antarmuka wajib berada di dalam direktori `unguided/` dengan rincian berkas berikut (satu file `.java` per kelas/interface):

```
None
unguided/
├── ItemTransaksi.java  (Interface)
├── TiketBioskop.java   (Abstract Class)
├── TiketReguler.java   (Concrete Subclass)
├── TiketVIP.java       (Concrete Subclass)
├── PaketSnack.java     (Concrete Class)
└── Main.java           (Kelas Eksekusi Utama)
```

  Pastikan baris paling atas di setiap berkas mencantumkan package yang sesuai (contoh: `package com.[nama_panggilan].unguided;`).

- **Enkapsulasi:** Seluruh *field* (atribut) wajib berstatus `private` dan hanya boleh diakses melalui *getter* oleh kelas di luar dirinya.
- **Keyword this:** Gunakan parameter dengan nama yang sama dengan nama *field* pada *constructor*, sehingga wajib menggunakan kata kunci `this` secara eksplisit.
- **Koleksi Statis:** Dilarang menggunakan tipe data koleksi dinamis seperti `ArrayList`. Gunakan array statis standar.

Soal — Sistem Kasir Terpadu Bioskop (*Cinema POS*)

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

Tugas 1: Fondasi Kelas Abstrak (*Abstract Class*)

Buatlah hierarki kelas tiket bioskop dengan spesifikasi berikut:

1. Kelas Induk `TiketBioskop` (Wajib `abstract`)

- **Atribut (Private):** `judulFilm` (`String`), `hargaDasar` (`double`), `jumlahKursi` (`int`).
- **Konstruktor:** Menerima dan menginisialisasi ketiga atribut tersebut.
- **Method:** Buat *getter* untuk ketiga atribut.
- **Abstract Method:**
  - `public abstract double hitungTotalBayar();`
  - `public abstract String getNamaTransaksi();` *(Beban perumusan kedua metode ini diserahkan kepada kelas turunannya agar output di struk nanti spesifik untuk tiap jenis tiket).*

2. Kelas Turunan `TiketReguler`

- **Atribut Tambahan (Private):** `nomorStudio` (`int`).
- **Konstruktor:** Menginisialisasi atribut induk via `super(judulFilm, hargaDasar, jumlahKursi)` dan atribut lokal `nomorStudio`.
- **Override Method:**
  - `hitungTotalBayar()`: Mengembalikan hasil `hargaDasar * jumlahKursi`.
  - `getNamaTransaksi()`: Mengembalikan string `"Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")"`.

3. Kelas Turunan `TiketVIP`

- **Atribut Tambahan (Private):** `biayaFasilitasVIP` (`double`). *(Catatan: Biaya fasilitas seperti sofa recliner ini dikenakan untuk **setiap kursi**).*
- **Konstruktor:** Menginisialisasi atribut induk via `super(...)` dan atribut lokal `biayaFasilitasVIP`.
- **Override Method:**
  - `hitungTotalBayar()`: Mengembalikan hasil `(hargaDasar + biayaFasilitasVIP) * jumlahKursi`.
  - `getNamaTransaksi()`: Mengembalikan string `"Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")"`.

Tugas 2: Kontrak Antarmuka Lintas Entitas (*Interface*)

Sistem kasir membutuhkan standar publik agar setiap barang bisa diproses.

1. Antarmuka `ItemTransaksi` Buatlah *interface* bernama `ItemTransaksi` dengan dua deklarasi kontrak (metode abstrak):

- `double getNominalBayar();`
- `String getNamaTransaksi();`

2. Integrasi Kelas `TiketBioskop` ke Antarmuka

- Tambahkan `implements ItemTransaksi` pada deklarasi kelas `TiketBioskop`.
- **Penting (Alasan Arsitektural):** Metode `hitungTotalBayar()` adalah logika internal spesifik tiket, sedangkan `getNominalBayar()` adalah kontrak publik kasir. Wujudkan metode `getNominalBayar()` di dalam `TiketBioskop` yang isinya sekadar me-*return* nilai dari `hitungTotalBayar()`.
- *(Catatan: Anda tidak perlu menata ulang `getNamaTransaksi()` di sini karena ia sudah dideklarasikan sebagai metode abstrak pada Tugas 1).*

3. Kelas Baru: `PaketSnack` (Benda Non-Tiket) Buat kelas independen bernama `PaketSnack` yang **bukan** turunan `TiketBioskop`, tetapi mengimplementasikan `ItemTransaksi`.

- **Atribut (Private):** `namaPaket` (`String`), `jumlahPorsi` (`int`), `hargaPerPorsi` (`double`).
- **Konstruktor:** Menginisialisasi ketiga atribut.
- **Override Method:**
  - `getNominalBayar()`: Mengembalikan hasil `jumlahPorsi * hargaPerPorsi`.
  - `getNamaTransaksi()`: Mengembalikan string `"F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")"`.

Tugas 3: Modernisasi Antarmuka Java 8+

Suntikkan fitur tambahan ke antarmuka `ItemTransaksi` tanpa mengharuskan kelas-kelas di atas dimodifikasi ulang:

1. Default Method Tambahkan metode `default void cetakRincianItem()` yang bertugas mencetak format baris struk ke layar. Panggil metode milik antarmuka (`getNamaTransaksi` dan `getNominalBayar`) untuk mengambil data secara dinamis. Format wajib: `- [Nama Transaksi (Kuantitas x Harga Satuan)] | Total: Rp[Nominal Bayar]`

2. Static Method Tambahkan metode `static void catatLogKasir(String pesan)` yang mencetak teks: `[LOG SISTEM KASIR] [pesan]`

Tugas 4: Pengujian Polimorfisme di `Main`

Tuliskan alur berikut di dalam *method* `main`:

1. **Uji Proteksi Abstraksi:** Tuliskan baris kode instansiasi `TiketBioskop` (misal: `new TiketBioskop(...)`), lalu jadikan baris tersebut sebagai **komentar** (`//`).
2. **Buka Kasir:** Panggil metode statis `catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");` langsung melalui nama antarmukanya.
3. **Buat Objek Transaksi:**
   - 1 objek `TiketReguler` (Film `"Interstellar"`, harga dasar 40000, 2 kursi, Studio 3).
   - 1 objek `TiketVIP` (Film `"Dune: Part Two"`, harga dasar 60000, 2 kursi, biaya VIP 25000).
   - 1 objek `PaketSnack` (`"Combo Popcorn Caramel + 2 Soda"`, 1 porsi, harga 55000).
4. **Array Polimorfik:** Simpan ketiga objek tersebut ke dalam satu array heterogen bertipe `ItemTransaksi[]`.
5. **Pemrosesan Dinamis (Loop):**
   - Gunakan perulangan (*for* atau *for-each*) terhadap array tersebut.
   - Di dalam perulangan, panggil *default method* `cetakRincianItem()` **lewat masing-masing objek/elemen array**.
   - Hitung dan jumlahkan nominal bayar keseluruhan belanjaan secara akumulatif.
6. **Tutup Kasir:** Tampilkan grand total seluruh belanjaan, lalu panggil kembali metode statis `catatLogKasir("Transaksi #042 selesai diproses.");`.

Contoh Output yang Diharapkan

```
None
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Jawaban :

```
// nama file: ItemTransaksi.java

package unguided;

public interface ItemTransaksi {
    // Kontrak antarmuka
    double getNominalBayar();
    String getNamaTransaksi();

    // Default Method (Java 8+) untuk mencetak rincian item
    default void cetakRincianItem() {
        System.out.println(getNamaTransaksi() + " | Total: Rp" + getNominalBayar());
    }

    // Static Method (Java 8+) untuk mencatat log sistem kasir
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

```
// nama file: TiketBioskop.java

package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
```

```
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    // Abstract method spesifik tiket
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Realisasi kontrak getNominalBayar() dari ItemTransaksi
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

```
// nama file: TiketReguler.java
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }
```

```
    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

```
// nama file: TiketVIP.java
package unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        double hargaTotalPerKursi = getHargaDasar() + this.biayaFasilitasVIP;
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + hargaTotalPerKursi + ")";
    }
}
```

```
// nama file: PaketSnack.java
package unguided;
```

```
public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket
                + " (" + this.jumlahPorsi + " x Rp" + this.hargaPerPorsi + ")";
    }
}
```

```
// nama file: Main.java
        package unguided;

        public class Main {
            public static void main(String[] args) {
                // 1. Uji Proteksi Abstraksi: TiketBioskop bersifat abstrak sehingga tidak bisa diinstansiasi langsung
```

```
        // TiketBioskop tiketPalsu = new TiketBioskop("Interstellar", 40000, 2); // Error saat kompilasi

        // 2. Buka Kasir lewat method static interface
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat Objek Transaksi
        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000.0, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000.0, 2, 25000.0);
        PaketSnack paketSnack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1, 55000.0);

        // 4. Array Polimorfik Heterogen
        ItemTransaksi[] keranjang = { tiketReguler, tiketVIP, paketSnack };

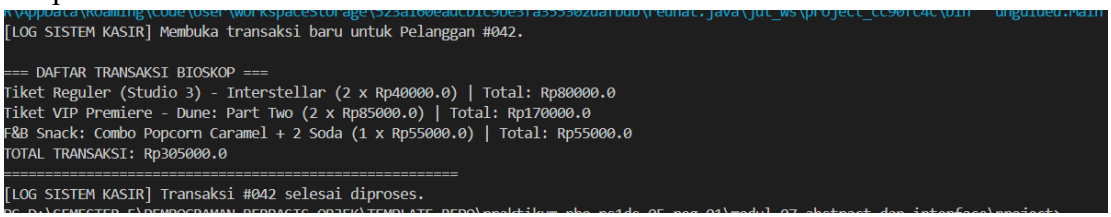
        // 5. Pemrosesan Dinamis (Loop)
        System.out.println("\n=== DAFTAR TRANSAKSI BIOSKOP ===");
        double totalTransaksi = 0.0;

        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("TOTAL TRANSAKSI: Rp" + totalTransaksi);
        System.out.println("============================================================");

        // 6. Tutup Kasir
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :



Penjelasan :

File ItemTransaksi.java merupakan interface kontrak kasir yang berisi deklarasi method getNominalBayar(), getNamaTransaksi(), serta fitur Java 8+ berupa default method cetakRincianItem() dan static method catatLogKasir().

File TiketBioskop.java adalah kelas abstrak induk bertipe ItemTransaksi yang memiliki atribut private judulFilm, hargaDasar, dan jumlahKursi, serta memetakan method getNominalBayar() ke method abstrak hitungTotalBayar().

File TiketReguler.java mewarisi TiketBioskop dengan atribut nomorStudio untuk meng-override hitungTotalBayar() (hargaDasar dikali jumlahKursi) dan format getNamaTransaksi().

File TiketVIP.java mewarisi TiketBioskop dengan atribut biayaFasilitasVIP untuk meng-override hitungTotalBayar() ((hargaDasar + biayaFasilitasVIP) dikali jumlahKursi) dan format getNamaTransaksi().

File PaketSnack.java mengimplementasikan interface ItemTransaksi secara independen untuk menghitung total harga snack (jumlahPorsi dikali hargaPerPorsi) dan format getNamaTransaksi().

File Main.java menjalankan transaksi kasir menggunakan array heterogen ItemTransaksi[], memanggil default method cetakRincianItem(), serta menghitung grand total pembayaran senilai Rp305.000,0.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa kelas abstract TiketBioskop digunakan sebagai kelas dasar yang tidak dapat diinstansiasi secara langsung dan mewajibkan kelas turunannya, yaitu TiketReguler dan TiketVIP, untuk mengimplementasikan method abstrak hitungTotalBayar(). Praktikum ini juga

menunjukkan penerapan interface ItemTransaksi sebagai kontrak publik yang mendefinisikan method getNominalBayar() dan getNamaTransaksi() untuk diimplementasikan oleh kelas abstrak maupun kelas independen seperti PaketSnack menggunakan keyword implements. Selain itu, penggunaan array heterogen bertipe ItemTransaksi[] berhasil menerapkan konsep polimorfisme untuk memproses berbagai jenis entitas transaksi dalam satu sistem kasir.

Praktikum ini juga membahas penggunaan default method cetakRincianItem() dan static method catatLogKasir() pada interface Java 8+ yang memungkinkan penambahan fungsi cetak struk dan pencatatan log sistem tanpa harus mengubah struktur kode pada kelas-kelas implementasinya. Perbedaan antara abstract class dan interface terlihat pada bagaimana kelas abstrak digunakan untuk merepresentasikan silsilah keluarga tiket bioskop, sedangkan interface digunakan sebagai kontrak umum untuk menyatukan item tiket dan non-tiket (snack). Dengan demikian, praktikum ini membantu memahami konsep abstraksi, antarmuka, dan polimorfisme heterogen serta penerapannya dalam perancangan sistem kasir terpadu menggunakan Java.